TEKNİK ARAŞTIRMA VE ALGORİTMİK MİMARİ RAPORU

TradingView Renko Grafiği Mekanikleri, Pine Script v5/v6 Alt Zaman Dilimi & Footprint Hesaplaması ve Python'da Birebir Yeniden Üretim

TradingView Resmî Yardım Merkezi Belgeleri ve Pine Script® v5/v6 Dil Başvuru Kılavuzu Temelinde Matematiksel Modelleme, Strateji Geri Test Yanılsamaları, Açık Kaynak Python Uygulamaları ve Belirsiz Noktaların Analizi

Bu kapsamlı teknik araştırma raporu; TradingView platformunun standart dışı fiyat tabanlı grafik motorlarından olan **Renko** grafiklerinin iç çalışma prensiplerini, Pine Script v5 ve v6 sürümlerindeki **`request.security_lower_tf()`** ve **`request.footprint()`** fonksiyonlarıyla gerçekleştirilen bar içi (intrabar) **Volume Delta / Footprint** hesaplamalarını ve bu yapıların **Python** ekosisteminde sıfır sapma ile yeniden üretilmesini incelemektedir.

Araştırma Ekseni	İncelenen Teknik Mekanizmalar	Temel Dayanak Kaynakları
1. Bölüm: TradingView Renko Mimarisi ve Geri Test Yanıltıcılığı	Kutu boyu atama yöntemleri (Traditional, ATR, Percentage LTP), Wilder RMA tabanlı ATR periyot ve bar kilitleme mekanizması, **`Close`** vs **`OHLC`** kaynak seçimi, ters yönlü fitil (wick) oluşumu, projeksiyon barları ve Broker Emülatörünün sentetik OHLC / 4-işlem varsayımından doğan geri test (backtest) yanılsamaları.	Understanding Renko Charts, What Do Renko Wicks Mean?, Strategy Peeking Into Future
2. Bölüm: Pine Script v5/v6 LTF ve Footprint / Volume Delta	**`request.security_lower_tf()`** dizi mimarisi, 100K/125K/200K intrabar sınırları, 3 kademeli alış/satış polarite durum makinesi, **`TradingView/ta`** kütüphanesi ve Pine Script v6 yerel **`request.footprint()`**, **`footprint`** ve **`volume_row`** nesneleri (POC, VAH/VAL eşitlik bozma kuralları ve %300 diyagonal imbalans).	Other Timeframes and Data, Pine v6 Type System, Volume Delta, Volume Footprint Guide
3. Bölüm: Python Açık Kaynak Ekosistemi ve Referans Kodlar	**`PyneSys/pynecore`**, **`srlcarlg/renkodf`**, **`ChillarAnand/stocktrends`** ve **`mplfinance`** paketlerinin TradingView uyumluluk denetimi; Wilder RMA ATR, OHLC fitilli Renko, LTF Volume Delta ve Footprint (POC, VA, Imbalance) için üretim seviyesinde Python kodları.	PyneCore Compatibility, renkodf, stocktrends, mplfinance
4. Bölüm: Resmi Belgelerde Belirsiz Kalan Noktalar	TradingView'ın kapalı kaynaklı (proprietary) arka uç motorunda açıkça belgelenmemiş 8 kritik algoritmik uç durum (edge case) ve bu belirsizliklerin kantitatif modellemeye etkileri.	Non-Standard Charts Data, Pine Script Repainting

Belge Türü: Kantitatif Mimarî ve Algoritmik Doğrulama Raporu	**Kapsanan Pine Script Sürümleri:** Pine Script® v5 ve v6 (2026 Güncel)
Dil ve Standart: Türkçe Teknik Referans Belgesi	**Tarih:** Ekim 2026

1. TradingView Renko Grafiği Mimarisi, Kurulumu ve Kutu Boyu Hesaplama Yöntemleri

Geleneksel mum (candlestick) ve çubuk (bar) grafikleri, zaman eksenini eşit aralıklı dilimlere bölerek her zaman dilimi içerisindeki açılış (**`open`**), en yüksek (**`high`**), en düşük (**`low`**) ve kapanış (**`close`**) fiyatlarını kaydeder. Buna karşın adını Japoncada "tuğla" anlamına gelen *renga* kelimesinden alan **Renko grafikleri**, zaman ve hacim boyutlarını grafik oluşum denkleminden tamamen çıkararak yalnızca fiyat hareketinin büyüklüğünü ölçer (Understanding Renko charts). TradingView grafik motorunda yeni bir Renko tuğlasının (brick) çizilebilmesi için fiyatın, en son tamamlanan tuğlanın üst veya alt kenarından kullanıcı veya algoritma tarafından belirlenen **kutu boyu (Box Size = B)** kadar uzaklaşması gerekir. Fiyat değişimi bu eşik değerin altında kaldığı müddetçe, piyasada saatler veya günler geçse dahi yeni bir tuğla oluşturulmaz.

TradingView resmi belgelerine göre Renko tuğlalarının grafik düzlemine yerleştirilmesinde iki mutlak geometrik kural uygulanır (Understanding Renko charts):

1. **Köşe Teması Kuralı:** Ardışık her Renko tuğlasının köşeleri daima birbirine temas eder (yeni tuğlanın sentetik açılış fiyatı, bir önceki tuğlanın üst veya alt sınırına kilitlenir).
2. **Tekil Dikey Sütun Kuralı:** Hiçbir dikey zaman/indeks sütununda aynı anda birden fazla Renko tuğlası bulunamaz; aynı zaman mumu içerisinde birden fazla kutu boyu büyüklüğünde fiyat hareketi gerçekleşse bile üretilen her tuğla yatay eksende ayrı bir sütuna yerleştirilir.

1.1 TradingView Arayüzünde ve Pine Script'te Renko Kurulumu

TradingView Supercharts arayüzünde bir Renko grafiği kurmak için üst araç çubuğundaki grafik türleri menüsünden **Renko** seçilir ve **Grafik Ayarları (Chart Settings) → Sembol (Symbol)** sekmesi üzerinden hesaplama parametreleri yapılandırılır (Understanding Renko charts). Arayüzde sunulan temel yapılandırma parametreleri şunlardır:

- **Kaynak (Source):** Tuğla hesaplamasında baz alınan zaman dilimi mumlarının yalnızca **`Close`** (Kapanış) fiyatlarının mı yoksa **`OHLC`** (Açılış, Yüksek, Düşük, Kapanış) değerlerinin mi kullanılacağını belirler (Understanding Renko charts).
- **Kutu Boyu Atama Yöntemi (Box size assignment method):** Tuğla yüksekliğinin hangi matematiksel modelle belirleneceğini seçer: **`Traditional`** (Geleneksel sabit mutlak değer), **`ATR`** (Average True Range - Ortalama Gerçek Aralık, varsayılan periyot **14**) veya **`Percentage (LTP)`** (Son işlem fiyatının yüzdesi, varsayılan **%1**) (Understanding Renko charts, Custom chart intervals).
- **Fitiller (Wicks):** Varsayılan olarak etkindir. Bir tuğla oluşmadan önce ters yönde gerçekleşen ancak dönüş eşiğini ($2 \times B$) aşamayan gerçek fiyat salınımlarını tuğla üzerinde iğne olarak gösterir (What do Renko wicks mean?).
- **Projeksiyon Barları (Projection Up / Down Bars):** Seçili grafik zaman dilimi (örneğin 15 dakikalık mum) henüz kapanmamışken, anlık fiyata dayalı olarak oluşabilecek potansiyel tuğlaları farklı bir renkle görselleştirir (Understanding Renko charts).

Pine Script v5 ve v6 ortamında ise herhangi bir grafik türü üzerinde çalışırken Renko fiyat serilerini programatik olarak talep etmek için **`ticker.renko()`** fonksiyonu ile özel bir sembol kimliği (ticker ID) oluşturulur ve bu kimlik **`request.security()`** fonksiyonuna argüman olarak geçirilir (Non-standard charts data):

```
//@version=6
indicator("Pine Script v6 Renko Veri Çekimi", overlay = true)

// "ATR" yöntemi (14 periyot) veya "Traditional" yöntemi (sabit kutu boyu, örn. 2.5)
string renkoATRId  = ticker.renko(syminfo.tickerid, "ATR", 14)
string renkoTradId = ticker.renko(syminfo.tickerid, "Traditional", 2.5)

[rOpen, rHigh, rLow, rClose] = request.security(renkoATRId, timeframe.period, [open, high, low, close])
plot(rClose, "Renko Kapanış (ATR 14)", color = color.rgb(2, 132, 199), linewidth = 2)
```

1.2 Kutu Boyu Yöntemleri: Traditional, ATR ve Percentage (LTP) Karşılaştırması

TradingView'ın sunduğu üç kutu boyu hesaplama yöntemi, volatiliteye uyum sağlama kabiliyeti ile tarihsel kararlılık (yeniden boyamama / non-repainting) arasında doğrudan bir mühendislik ödünleşimi (trade-off) barındırır (Understanding Renko charts, Custom chart intervals):

Kutu Boyu Yöntemi	Matematiksel Hesaplama Modeli	Varsayılan Değer	Tarihsel Kararlılık, Yeniden Boyama (Repainting) ve Kullanım Karakteristiği
Traditional (Geleneksel)	Kullanıcı tarafından tanımlanan sabit mutlak fiyat adımı: $B = Sabit$ (ör. 2.00 puan veya enstrüman değerinin yaklaşık 1/20'si).	Kullanıcı tanımlı mutlak sayı	**Tarihsel Olarak Sabit (Non-Repainting Grid):** Kutu boyu zamanla değişmediği için kapanmış barlardan oluşan geçmiş Renko tuğlaları grafik yenilendiğinde asla değişmez (Understanding Renko charts). Kontrollü strateji deneyleri için tek deterministik yöntemdir (ATR Based Renko Brick Size).
ATR (Ortalama Gerçek Aralık)	Baz alınan standart OHLC mum grafiği üzerinde hesaplanan n periyotluk Wilder RMA-ATR değerini kutu boyu olarak atar: $B = ATR_n$.	**`ATR Length = 14`**	**Dinamik Kutu / Tarihsel Repainting:** Volatilite değiştiğinde son ATR değeri güncellenir ve tüm geçmiş Renko tuğlaları yeni ATR kutu boyuna göre baştan inşa edilir (Understanding Renko charts, LuxAlgo Renko).
Percentage (LTP)	Kullanıcı yüzdesini (p) en son kapanış fiyatına (C_{last}) uygular ve **`mintick`**'e yuvarlar: $B = round((p / 100) \cdot C_{last}, mintick)$.	**`1%`** (%1)	**Tam Repainting (Açıkça Belgelenmiş):** TradingView resmi kılavuzu bu yöntemin en son kapanış fiyatını tüm geçmiş barlara sabit olarak uyguladığını ve *"yeniden boyamaya (repainting) tabi olduğunu"* açıkça belirtir (Understanding Renko charts).

1.3 ATR'nin Hangi Periyottan, Hangi Bardan ve Nasıl Hesaplandığı

Kullanıcı kutu boyu yöntemini **ATR** olarak seçtiğinde TradingView'ın arka planda yürüttüğü hesaplama üç temel kurala dayanır (Understanding Renko charts, To Pine Script version 5):

1. **Hangi Grafik ve Periyot Üzerinden Hesaplanır?** ATR, Renko tuğlalarının kendi sentetik fiyatları üzerinden değil, **grafikte o anda seçili olan baz zaman diliminin (örneğin 5 dakika, 1 saat veya 1 gün) standart OHLC mum grafiği (regular candlestick chart) üzerinden** hesaplanır (Understanding Renko charts). Varsayılan geri bakış periyodu **14**'tür (**`ATR Length = 14`**).
2. **Hangi Yumuşatma Formülü Kullanılır?** Pine Script'in yerleşik **`ta.atr(length)`** fonksiyonu ile birebir aynı şekilde, önce her standart mum için Gerçek Aralık ($TR_t = max(H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|)$) bulunur; ardından bu seri **Wilder'ın RMA (Running Moving Average, $\alpha = 1 / n$)** yöntemiyle yumuşatılır (To Pine Script version 5, Debugging RMA):

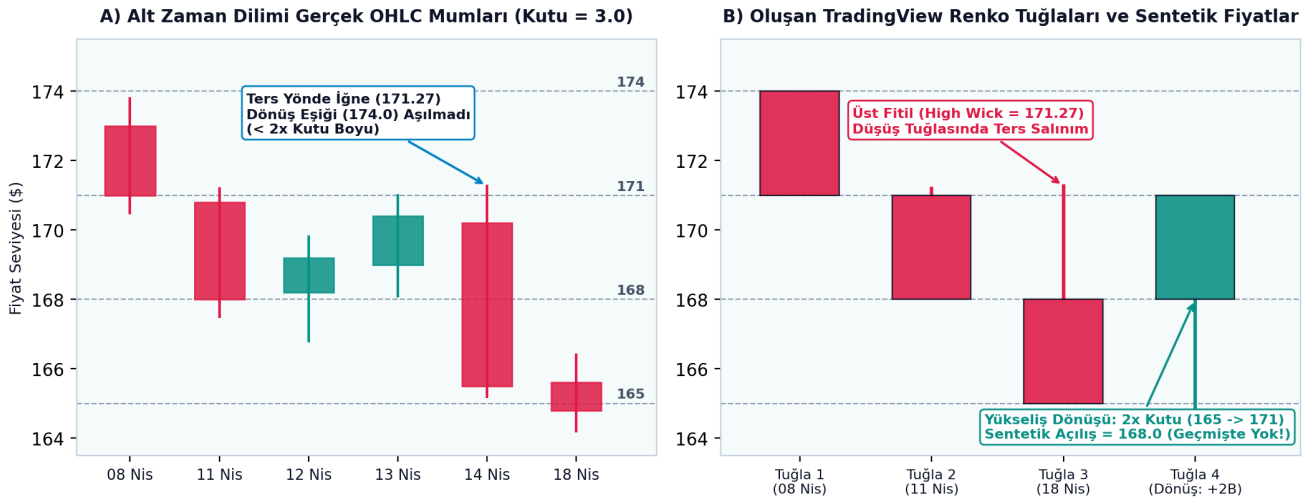
$$ATR_t = (1 / n) \cdot TR_t + ((n - 1) / n) \cdot ATR_{t-1} \quad (\text{İlk değer: } ATR_n = (1 / n) \sum_{i=1..n} TR_i)$$

3. **Hangi Barın ATR Değeri Kutu Boyu Yapılır? (Küresel Yeniden İnşa Kuralı):** TradingView, geçmişteki her bir Renko tuğlasını o günkü yerel ATR_t değeriyle farklı boyutlarda çizmez; **baz alınan standart mum grafiğinin en son (güncel) barındaki ATR değerini alır ve bu tekil değeri tüm grafik geçmişindeki tuğlalara sabit kutu boyu olarak uygular** (Understanding Renko charts, LuxAlgo Renko). Dolayısıyla standart grafikte her yeni mum kapandığında ATR değeri ufak da olsa değişir ve grafik yenilendiğinde (reload) geçmişteki tüm Renko tuğlalarının sınırları ve sayısı baştan aşağı değişir (LuxAlgo Renko).

1.4 Kutu Oluşumunda Kapanış mı Yüksek/Düşük mü Kullanılır? Fitil (Wick) Matematiği

İdeal bir Renko grafiği her bir tekil işlem (tick) verisi üzerinden inşa edilir; ancak TradingView, Renko tuğlalarını oluşturmak için **grafiğin seçili zaman dilimindeki (resolution) mumların `Close` (Kapanış) veya `OHLC` (Açılış, Yüksek, Düşük, Kapanış) değerlerini** kullanır (Understanding Renko charts). Örneğin 10 dakikalık bir Renko grafiğinde tuğlalar yalnızca 10 dakikalık mum kapandığında kalıcı olarak geçmişe kilitlenir (Understanding Renko charts). Dolayısıyla seçilen zaman dilimi ne kadar küçülürse (ör. 1 dakika veya 1 saniye), üretilen Renko tuğlaları gerçek tick tabanlı Renko'ya o kadar yaklaşır (Understanding Renko charts).

- **Kaynak = `Close` Seçildiğinde:** Yalnızca alt zaman dilimi mumunun kapanış fiyatı (C_t) dikkate alınır. Mum içinde fiyat en yüksek (H_t) veya en düşük (L_t) seviyelerde kutu eşiğini aşmış olsa bile, mum kapanışı eşik altında kalırsa yeni tuğla oluşmaz (Understanding Renko charts).
- **Kaynak = `OHLC` Seçildiğinde:** Alt zaman dilimi mumunun **`High`** ve **`Low`** değerleri de tuğla üretiminde ve fitil hesabında kullanılır (What do Renko wicks mean?).
- **Aynı Yönde *1 × B* ve Ters Yönde *2 × B* Kuralı:** Son tuğla ile aynı yönde yeni bir tuğla oluşması için fiyatın son tuğla sınırını *1 × B* aşması yeterlidir. Ancak ters yönde bir dönüş tuğlası oluşabilmesi için fiyatın **önce son tuğlanın açılış seviyesine geri dönmesi (*1 × B*), ardından bu seviyenin ötesine bir kutu boyu daha (*1 × B*) geçerek toplamda *2 × B* hareket etmesi** şarttır (What do Renko wicks mean?).
- **Fitillerin (Wicks) Oluşum Kuralı:** Bir önceki tuğlanın oluştuğu bardan mevcut tuğlanın tamamlandığı bara kadar geçen süre zarfında, fiyat son tuğlanın tersi yönde ***1 × B*'den fazla ancak *2 × B*'den az** hareket edip (yani önceki tuğlanın açılışını geçip dönüşü tamamlayamadan) tekrar ana yönde yeni bir tuğla oluşturursa, bu başarısız dönüş salınımı **fitil (wick)** olarak çizilir (What do Renko wicks mean?). Bu nedenle **Üst Fitiller (High Wicks) yalnızca düşüş tuğlalarında**, **Alt Fitiller (Low Wicks) ise yalnızca yükseliş tuğlalarında** görülür (What do Renko wicks mean?). Tuğlaların **`Open`** ve **`Close`** değerleri sentetik (kutu boyunun tam katı) iken, fitil uçları alt grafikteki gerçek **`Close`** veya **`High/Low`** fiyatlarını gösterir (What do Renko wicks mean?).

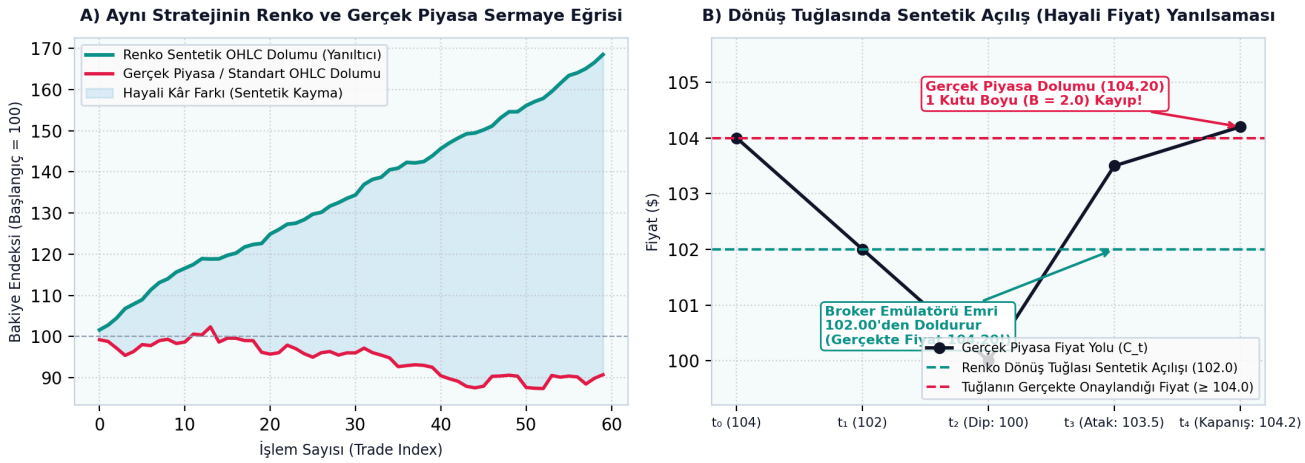


Şekil 1: TradingView resmi belgelerindeki AAPL (1G, OHLC, Kutu Boyu = 3.0) örneğinin mekanik gösterimi (What do Renko wicks mean?). Sol panelde 11 Nisan sonrası fiyat 171.27'ye yükselerek önceki tuğla açılışını (171.0) aşmış ancak 2x dönüş eşiği olan 174.0'a ulaşamadan 165.0'in altına inmiştir. Sağ panelde bu hareket 18 Nisan düşüş tuğlası üzerinde 171.27 seviyesinde gerçek bir **Üst Fitil (High Wick)** olarak kaydedilmiştir.

1.5 Strateji Geri Testinde (Backtesting) Renko'nun Bilinen Yanıltıcılığı

TradingView Pine Script Kullanıcı Kılavuzu ve Destek Merkezi belgeleri, Renko grafikleri üzerinde yapılan strateji geri testlerinin (Strategy Tester) **gerçek piyasa koşullarını yansıtmadığını ve sistematik olarak gerçek dışı kârlılık ürettiğini** açıkça belgelemektedir (Concepts / Strategies, Strategy produces unrealistically good results by peeking into the future, Concepts / Repainting). Bu yanıltıcılığın 5 matematiksel nedeni aşağıda özetlenmiştir:

1. **Sentetik Açılış Fiyatından (Synthetic Open) Emir Dolumu:** Bir düşüş trendinin ardından [100, 102] aralığındaki son düşüş tuğlasından sonra yükseliş yönlü bir dönüş tuğlası oluşabilmesi için gerçek piyasa fiyatının 104.00 seviyesine ($2 \times B$) ulaşması gerekir. Ancak oluşan yeni yükseliş tuğlası [102, 104] aralığında çizilir; yani tuğlanın sentetik açılış fiyatı (**open**) 102.00'dir! Broker Emülatörü bu tuğlanın açılışında bir alış emri doldurduğunda, gerçekte fiyat 104.00 iken işlemi **geçmişte kalmış sentetik 102.00 fiyatından** gerçekleştirir ve her dönüş işleminde $1 \times B$ kadar hayali kâr yazar (Strategy Peeking Into Future).
2. **Broker Emülatörünün 4 İşlem (Open → High/Low → Close) Varsayımı ve Geleceği Görme (Peeking):** TradingView Strateji Motoru her barı 4 işlem noktası olarak simüle eder (Concepts / Strategies, Strategy Peeking Into Future). Renko kutu boyutu çok küçük (özellikle **`mintick`** altı) seçildiğinde veya tek bir gerçek mum içinde çoklu tuğla oluştuğunda, strateji gerçek fiyat işlenmeden önce bir sonraki tuğlanın yönünü "geleceğe bakarak" tespit eder ve tepe/dip noktalarından işlem yapar (Strategy Peeking Into Future).
3. **`fill_orders_on_standard_ohlc = true` Özelliğinin Renko'yu Desteklememesi:** Pine Script'teki **`fill_orders_on_standard_ohlc = true`** parametresi yalnızca **Heikin Ashi** grafiklerinde emirlerin gerçek piyasa OHLC fiyatlarından doldurulmasını sağlar (Concepts / Strategies). Renko tuğlalarının zaman ekseni standart mumlarla birebir eşleşmediği (bir zaman mumunda sıfır veya birden fazla tuğla oluşabildiği) için Renko grafiklerinde bu koruma mekanizması çalışmaz (Concepts / Strategies).
4. **Zaman Sıkıştırması ve Bar İçi Projeksiyon Kaybı:** Canlı piyasada bar kapanmadan önce oluşan projeksiyon tuğlaları (**`Projection bricks`**) stratejiyi tetikleyebilir; ancak mum kapanışında fiyat geri çekilirse bu tuğlalar tarihsel grafikten silinir (Understanding Renko charts).
5. **ATR / Percentage Yeniden Boyaması:** ATR veya Percentage kutu boyu kullanıldığında geçmiş tuğlalar her yeni volatilite rejiminde yeniden çizildiği için geri test sonuçları zamana göre sürekli değişir (Understanding Renko charts, LuxAlgo Renko).



Şekil 2: Renko grafiğinde strateji geri testi yanılsamasının mekaniği (Concepts / Strategies, Strategy Peeking Into Future). Sol panelde aynı sinyallerin sentetik Renko OHLC dolumu ile gerçek piyasa OHLC dolumu arasındaki kümülatif sermaye farkı; sağ panelde ise $2 \times B$ dönüş tuğlasında fiyat gerçekte 104.20 iken Broker Emülatörünün emri 102.00 sentetik açılışından doldurarak yarattığı hayali kazanç görülmektedir.

---

2. Pine Script v5/v6: `request.security_lower_tf()` ve Footprint / Volume Delta Hesaplaması

Pine Script v5 ve v6 sürümlerinde grafiğin çalıştığı zaman diliminin (**`timeframe.period`**) altındaki bar içi (intrabar) fiyat ve hacim hareketlerini analiz etmek için iki temel mimari mevcuttur: her grafik barı için alt zaman dilimi değerlerini sıralı bir dizi (**`array<type>`**) olarak döndüren **`request.security_lower_tf()`** fonksiyonu ve Mart 2026'da Pine Script v6'ya eklenen yerel nesne tabanlı **`request.footprint()`** mimarisi (Concepts / Other timeframes and data, Volume footprints in Pine scripts).

2.1 `request.security_lower_tf()` Mimarisi, Çalışma Kuralları ve Sınırları

Standart **`request.security()`** fonksiyonu alt zaman dilimi için çağrıldığında yalnızca tek bir intrabar değerini döndürürken ve gerçek zamanlı (realtime) barlarda sıralanmamış veri nedeniyle yeniden boyamaya (repainting) yol açarken, **`request.security_lower_tf()`** fonksiyonu **ana grafik barının kapsadığı tüm alt zaman dilimi mumlarını (intrabars) açılış zamanlarına göre artan sırada (ascending order) bir Pine dizisi (`array<type>`) içinde döndürür** (Concepts / Other timeframes and data, Concepts / Repainting).

```
request.security_lower_tf(symbol, timeframe, expression, ignore_invalid_symbol, currency,
ignore_invalid_timeframe, calc_bars_count) → array<type>
```

- **Dönüş Tipi ve Demet (Tuple) Desteği:** **`expression`** argümanına tekil bir seri (ör. **`close`**) verildiğinde **`array<float>`** döner. Eğer **`[open, high, low, close, volume]`** gibi bir tuple verilirse, fonksiyon her bir eleman için ayrı bir dizi referansı içeren **`[array<float>, array<float>, ...]`** tuple'ı döndürür (Concepts / Other timeframes and data). Bir betikteki tüm **`request.*()`** çağrılarının toplam tuple elemanı sayısı **127**'yi geçemez (Writing / Limitations).
- **Plan Bazlı Intrabar Geçmiş Sınırları:** Her bir **`request.security_lower_tf()`** çağrısı en son **100.000** (Basic, Essential, Plus, Premium planlar), **125.000** (Expert plan) veya **200.000** (Ultimate plan) intrabar çekebilir (Writing / Limitations). Örneğin 60 dakikalık bir grafikte 1 dakikalık veri çekildiğinde, her saatlik barda en fazla 60 intrabar bulunur ve 100.000 bar sınırı en az *100.000 / 60 ≈ 1.666* saatlik grafik barını kapsar (Writing / Limitations). Daha eski tarihsel barlarda fonksiyon **boş dizi (`array.size() == 0`)** döndürür (Concepts / Other timeframes and data).
- **Pine Script v5 vs v6 Dinamik İstek Farkı:** Pine Script v5'te varsayılan olarak kapalı olan **`dynamic_requests`**, Pine Script v6'da **varsayılan olarak etkindir** (Concepts / Other timeframes and data). Böylece v6'da **`request.security_lower_tf()`** döngü ve koşul blokları içinde doğrudan çağrılabilir.

2.2 TradingView Yerleşik `Volume Delta` Algoritması ve LTF Kodlaması

TradingView'ın yerleşik **Volume Delta** indikatörü (ve resmi **`TradingView/ta`** kütüphanesindeki **`requestVolumeDelta()`** / **`requestUpAndDownVolume()`** fonksiyonları), her bir grafik barını alt zaman dilimi (intrabar) mumlarına bölerek her intrabarın hacmini pozitif (alış) veya negatif (satış) olarak sınıflandırır (Volume Delta).

Ana Grafik Zaman Dilimi	Volume Delta Otomatik LTF	Volume Footprint Otomatik LTF Kademeli Geçiş Zinciri
Saniye Grafikleri (`Seconds`)	**`1S`** (1 Saniye)	**`1T`** (Profesyonel planlar için 1 Tick) → **`1S`** (1 Saniye)
Dakika ve Saat Grafikleri (`Minutes / Hours`)	**`1`** (1 Dakika)	**`1T`** → **`1S`** → **`1`** (1 Dakika) → **`60`** (60 Dakika)
Günlük Grafikler (`Daily`)	**`5`** (5 Dakika)	**`1`** (1 Dakika) → **`60`** (60 Dakika)
Haftalık, Aylık ve Diğerleri (`Others`)	**`60`** (60 Dakika)	**`60`** (60 Dakika)

Resmî Intrabar Hacim Polaritesi (Alış / Satış) Sınıflandırma Algoritması: Hem *Volume Delta* hem de *Volume Footprint* hesaplamalarında her bir i alt zaman dilimi mumu için alış (`+1`) veya satış (`-1`) yönü şu 3 kademeli kuralla tayin edilir (Volume Delta, Volume footprint charts: a complete guide):

1. **Açılış ve Kapanış Farklıysa ($C_i \neq O_i$):** Eğer $C_i > O_i$ ise intrabar hacmi **Alış (Pozitif, $+V_i$)**; eğer $C_i < O_i$ ise intrabar hacmi **Satış (Negatif, $-V_i$)** olarak sınıflandırılır.
2. **Açılış ve Kapanış Eşitse ($C_i = O_i$ ve $C_i \neq C_{i-1}$):** Mevcut intrabarın kapanışı bir önceki intrabarın kapanışı (C_{i-1}) ile karşılaştırılır; $C_i > C_{i-1}$ ise **Alış ($+V_i$)**, $C_i < C_{i-1}$ ise **Satış ($-V_i$)** atanır.
3. **Tam Eşitlik Durumu ($C_i = O_i$ ve $C_i = C_{i-1}$):** Mevcut intrabara, **bir önceki intrabarın sahip olduğu pozitif veya negatif kategori (s_{i-1}) aynen devredilir** (Volume Delta).

Ana grafik barı boyunca polarize edilmiş intrabar hacimleri adım adım toplanarak kümülatif seri oluşturulur: Volume Delta mumunun **Açılışı (`Open`) daima 0'dır**; **Kapanışı (`Close`)** bar sonundaki net toplam deltayı; **En Yüksek (`High`)** ve **En Düşük (`Low`)** değerleri ise bar süresi boyunca ulaşılan maksimum ve minimum kümülatif delta seviyelerini gösterir (Volume Delta). Aşağıda bu algoritmanın Pine Script v6'da **`request.security_lower_tf()`** ile tam gerçeklemesi yer almaktadır:

```
//@version=6
indicator("TradingView Volume Delta (LTF Birebir Uygulama)", format = format.volume)

bool useCustomTF = input.bool(false, "Özel Alt Zaman Dilimi Kullan")
string customTF  = input.timeframe("1", "Alt Zaman Dilimi")

// 1. TradingView Resmi Otomatik LTF Seçim Kuralı
string ltf = switch
    useCustomTF          => customTF
    timeframe.isseconds  => "1S"
    timeframe.isintraday => "1"
    timeframe.isdaily    => "5"
    => "60"

// 2. 3 Kademeli Intrabar Polarite Durum Makinesi
upAndDownVolume() =>
    float posVol = 0.0
    float negVol = 0.0
    var int prevPol = 1
    int pol = switch
        close > open     =>  1
        close < open     => -1
        close > close[1] =>  1
        close < close[1] => -1
        => prevPol
    prevPol := pol
    if pol == 1
        posVol := volume
    else
        negVol := -volume
    [posVol, negVol]

// 3. Alt Zaman Dilimi Dizilerinin Çekilmesi ve Kümülatif Delta Mumu İnşası
[posArr, negArr] = request.security_lower_tf(syminfo.tickerid, ltf, upAndDownVolume())

float dOpen = 0.0, float cumD = 0.0, float maxD = 0.0, float minD = 0.0
int count = array.size(posArr)
if count > 0
    for i = 0 to count - 1
        cumD += array.get(posArr, i) + array.get(negArr, i)
        maxD := math.max(maxD, cumD)
        minD := math.min(minD, cumD)

float dClose = count > 0 ? cumD : na
color cColor = dClose >= 0 ? color.teal : color.maroon
plotcandle(dOpen, count > 0 ? maxD : na, count > 0 ? minD : na, dClose, "Volume Delta", color = cColor,
wickcolor = cColor, bordercolor = cColor)
```

2.3 Pine Script v6 Yerel `request.footprint()`, `footprint` ve `volume_row` Mimarisi

Pine Script v6'ya Mart 2026'da eklenen **`request.footprint()`** fonksiyonu, **`request.security_lower_tf()`** ile manuel dizi döngüsü kurmaya gerek kalmadan doğrudan sunucu tarafında hesaplanmış **Volume Footprint (Hacim İzi)** nesnesini döndürür (Concepts / Other timeframes and data, Language / Type system, Volume footprints in Pine scripts):

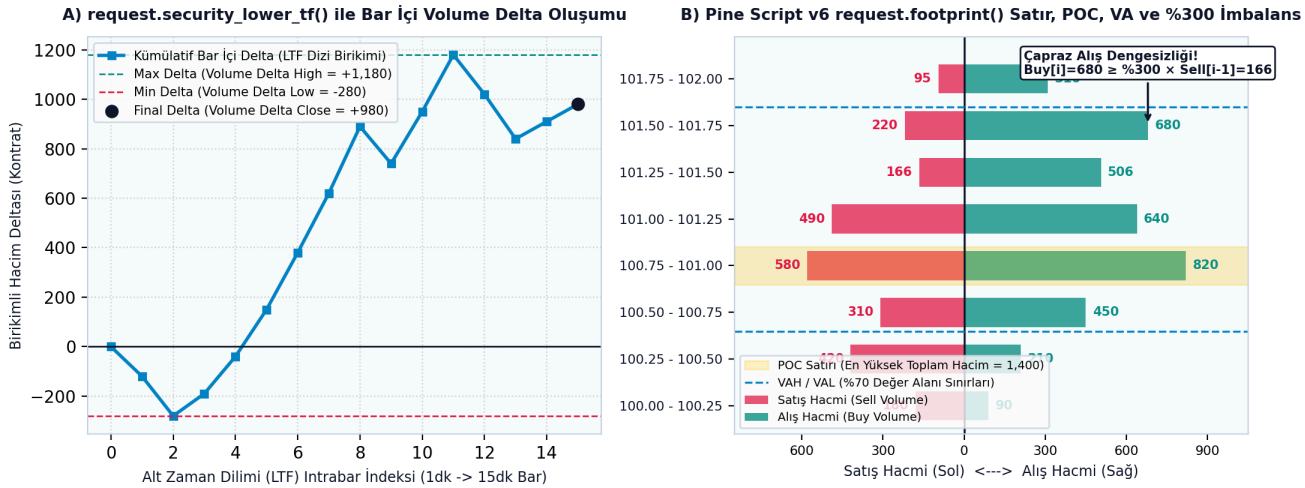
```
request.footprint(ticks_per_row, va_percent, imbalance_percent) → series footprint
```

Fonksiyon yalnızca **Premium** ve **Ultimate** abonelik planlarında çalışır ve bir betik (yardımcı `request.security` bağlamları dahil) **en fazla 1 adet benzersiz `request.footprint()` çağrısı** yürütebilir (Writing / Limitations). Döndürülen `footprint` ve `volume_row` nesnelerinin yerleşik metodları şunlardır (Language / Type system):

Pine Script v6 Metod İmzası	Dönüş Tipi	Matematiksel ve Yapısal İşlevi
`footprint.buy_volume(id)` / `sell_volume(id)`	`series float`	Barın tüm fiyat satırlarındaki toplam alış (yukarı yönlü) ve toplam satış (aşağı yönlü) hacimlerini döndürür.
`footprint.total_volume(id)` / `delta(id)`	`series float`	Barın toplam hacmini ($V_{buy} + V_{sell}$) ve genel hacim deltasını ($V_{buy} - V_{sell}$) döndürür.
`footprint.poc(id)` / `vah(id)` / `val(id)`	`volume_row`	Sırasıyla Kontrol Noktası (POC - en yüksek hacimli satır), Değer Alanı Üst Sınırı (VAH) ve Değer Alanı Alt Sınırı (VAL) satırlarının `volume_row` referansını döndürür.
`footprint.rows(id)` / `get_row_by_price(id, p)`	`array<volume_row>` / `volume_row`	`rows()` tüm satırları en düşük fiyattan en yüksek fiyata sıralı dizi olarak döndürür; `get_row_by_price()` verilen fiyatı içeren satırı bulur.
`volume_row.up_price(id)` / `down_price(id)`	`series float`	İlgili satırın üst ve alt fiyat sınırlarını döndürür ($P_{up} - P_{down} = ticks_per_row \times syminfo.mintick$).
`volume_row.buy_volume(id)` / `sell_volume(id)` / `delta(id)`	`series float`	İlgili fiyat satırına ait alış hacmi, satış hacmi ve satır içi delta ($V_{buy,r} - V_{sell,r}$) değerlerini döndürür.
`volume_row.has_buy_imbalance(id)` / `has_sell_imbalance(id)`	`series bool`	**Çapraz (Diyagonal) İmbalans Kontrolü:** Alış dengesizliği için mevcut satırın alış hacminin **bir alt satırın satış hacminden**; satış dengesizliği için mevcut satırın satış hacminin **bir üst satırın alış hacminden** `imbalance_percent` (varsayılan %300 = 3 kat) büyük olup olmadığını denetler (Language / Type system, Volume footprint guide).

Değer Alanı (Value Area — VAH / VAL) ve POC Hesaplama Algoritması: TradingView resmi belgelerine göre hem *Volume Profile* hem de *Volume Footprint* hesaplamalarında Değer Alanı (varsayılan %70) şu deterministik adımlarla inşa edilir (Volume profile indicators: basic concepts, Volume footprint guide):

1. Bar içindeki tüm fiyat satırlarının toplam hacmi toplanır ve `va_percent` (varsayılan 0.70) ile çarpılarak hedef Değer Alanı hacmi ($V_{target} = 0.70 \times V_{total}$) belirlenir.
2. En yüksek toplam hacme sahip satır **Kontrol Noktası (Point of Control — POC)** olarak seçilir, Değer Alanına ilk satır olarak eklenir ve hacmi hedef hacimden düşülür.
3. Her adımda mevcut Değer Alanı sınırının hemen üstündeki aday satır ile hemen altındaki aday satırın toplam hacimleri karşılaştırılır ve hacmi daha büyük olan satır seçilir. Seçilen satırın hacmi kalan hedef hacmi aşmıyorsa Değer Alanına dahil edilerek üst sınır (VAH) veya alt sınır (VAL) güncellenir; eğer hedefi aşıyorsa algoritma durur (Volume profile basic concepts).
4. **Resmî Eşitlik Bozma Kuralları (Tie-Breaking Rules):** Üst ve alt aday satırların toplam hacimleri birbirine eşitse, **POC satırına (satır adımı olarak) daha yakın olan aday satır** seçilir. Eğer POC satırına olan uzaklıkları da eşitse, **POC satırının üstündeki (daha yüksek fiyatlı) satır** tercih edilir (Volume profile basic concepts).



Şekil 3: Sol panelde **`request.security_lower_tf()`** ile 15 dakikalık bir bar içindeki 1 dakikalık intrabarların kümülatif hacim deltası ilerleyişi (Open=0, High=+1180, Low=-280, Close=+980); sağ panelde ise Pine Script v6 **`request.footprint()`** fonksiyonunun fiyat satırları, POC satırı, %70 Değer Alanı (VAH/VAL) sınırları ve %300 çapraz alış dengesizliği ($Buy_r \geq 3 \times Sell_{r-1}$) görülmektedir.

```
//@version=6
indicator("Pine Script v6 Yerel Footprint & POC/VA/Imbalance Analizi", overlay = true)

int ticksPerRow = input.int(100, "Satır Başına Tick (ticks_per_row)", minval = 1)
float vaPct     = input.float(70.0, "Değer Alanı Yüzdesi (va_percent)", minval = 1.0, maxval = 100.0)
float imbPct    = input.float(300.0, "Dengesizlik Yüzdesi (imbalance_percent)", minval = 100.0)

footprint fp = request.footprint(ticksPerRow, vaPct, imbPct)

if not na(fp)
    float barDelta = fp.delta()
    volume_row pocRow = fp.poc()
    volume_row vahRow = fp.vah()
    volume_row valRow = fp.val()
    float pocMid = math.avg(pocRow.up_price(), pocRow.down_price())

    int buyImbCount = 0, int sellImbCount = 0
    for r in fp.rows()
        if r.has_buy_imbalance()
            buyImbCount += 1
        if r.has_sell_imbalance()
            sellImbCount += 1
    log.info("Delta: {0}, POC: {1}, BuyImb: {2}, SellImb: {3}", barDelta, pocMid, buyImbCount,
sellImbCount)
```

---

3. Python'da Birebir Yeniden Üretim: Açık Kaynak Örnekler ve Referans Uygulamalar

TradingView'ın Renko, **`request.security_lower_tf()`** ve Footprint / Volume Delta hesaplamalarını Python ortamında yeniden üretmek için mevcut açık kaynak kütüphanelerin yetenekleri ve TradingView motorundan ayrıştıkları noktalar aşağıda karşılaştırılmıştır:

Açık Kaynak Kütüphane	Kapsadığı Özellikler	TradingView Uyumluluk Düzeyi	Temel Kısıtlamalar ve Farklılıklar
PyneSys / PyneCore (GitHub, Docs)	Pine Script v6 Python çalışma zamanı, **`ta.*`** (67 fonksiyon), **`strategy.*`**, **`request.security()`** ve **`request.security_lower_tf()`**.	1.000 gerçek TradingView betiğinde 131,7 milyon değerde **%99,771 bit-özdeşliği**; **`request.security_lower_tf()`** ve **`ta.atr()`** (Wilder RMA) birebir aynıdır (PyneCore Compatibility).	Yola bağımlı sentetik grafiklerden yalnızca **`ticker.heikinashi()`** desteklenir; **`ticker.renko()`** **`NotImplementedError`** fırlatır (PyneCore Ticker). **`request.footprint()`** çevrimdışı tick kaynağı olmadığından **`na`** döner (PyneCore Request).
srlcarlg / renkodf (GitHub, PyPI)	Tick verisinden NumPy hızlandırmalı OHLCV Renko üretimi (**`Renko`** ve WebSocket için **`RenkoWS`** sınıfları).	**`'wicks'`** (varsayılan) ve **`'normal'`** modları sayesinde *1 × B* devam, *2 × B* dönüş ve ters yönlü fitil kurallarını doğru uygular (renkodf PyPI).	Yalnızca sabit **`brick_size`** ve tick/close sütunu kabul eder; TradingView'ın OHLC mum çözünürlüğü bazlı inşasını ve Wilder RMA-ATR / Percentage LTP kutu boyu yöntemlerini içermez.
ChillarAnand / stocktrends (GitHub, Context7)	Pandas OHLC verisi üzerinden **`indicators.Renko`**, **`LineBreak`** ve **`PnF`** hesaplaması.	**`Renko.PERIOD_CLOSE`** modu ile sabit kutu boyunda temel kapanış bazlı tuğlaları üretir (Stocktrends Docs).	Fitilleri (wicks), **`OHLC`** kaynak modunu, Wilder RMA tabanlı ATR kutu boyunu ve bar içi projeksiyon mantığını desteklemez.
matplotlib / mplfinance (GitHub)	**`mpf.plot(df, type='renko', renko_params=dict(brick_size='atr', atr_length=14))`** ile görselleştirme.	Sabit kutu boyu ve ATR tabanlı kutu boyu ile temel görsel Renko çizimi sağlar.	Fitilleri (wicks) hesaplamaz, yalnızca kapanış fiyatlarını kullanır ve ATR yuvarlaması TradingView'ın **`mintick`** ızgarasıyla birebir eşleşmeyebilir.

3.1 Python Referans Uygulaması 1: TradingView Birebir Uyumlu Renko Motoru

Aşağıdaki Python kodu; TradingView'ın **Traditional**, **Wilder RMA tabanlı ATR** ve **Percentage (LTP)** kutu boyu yöntemlerini, **`close`** ve **`ohlc`** kaynak seçimlerini, *1 × B* devam / *2 × B* dönüş kurallarını ve yalnızca ters yönde oluşan gerçek fiyat fitillerini (wicks) eksiksiz olarak yeniden üretmektedir:

```
import numpy as np
import pandas as pd
from typing import Literal

def tv_wilder_atr(df: pd.DataFrame, length: int = 14) -> pd.Series:
    '''TradingView Pine Script ta.atr(length) birebir karşılığı: True Range üzerinde Wilder RMA
(1/length).'''
    h = df["high"].to_numpy(dtype=np.float64)
    l = df["low"].to_numpy(dtype=np.float64)
    c = df["close"].to_numpy(dtype=np.float64)
    tr = np.empty(len(df), dtype=np.float64)
```

```
    tr[0] = h[0] - l[0]
    tr[1:] = np.maximum(h[1:] - l[1:], np.maximum(np.abs(h[1:] - c[:-1]), np.abs(l[1:] - c[:-1])))
    atr = np.full(len(df), np.nan, dtype=np.float64)
    if len(df) >= length:
        atr[length - 1] = np.mean(tr[:length])
        alpha = 1.0 / float(length)
        for i in range(length, len(df)):
            atr[i] = alpha * tr[i] + (1.0 - alpha) * atr[i - 1]
    return pd.Series(atr, index=df.index)

def build_tradingview_renko(
    df: pd.DataFrame,
    method: Literal["Traditional", "ATR", "Percentage"] = "Traditional",
    box_size: float = 1.0,
    atr_length: int = 14,
    pct_value: float = 1.0,
    source: Literal["close", "ohlc"] = "close",
    mintick: float = 0.01,
    wicks: bool = True
) -> pd.DataFrame:
    '''TradingView Renko Grafik Motorunun (Traditional, ATR, Percentage, Close/OHLC ve Fitiller) Birebir
Üretimi.'''
    if method == "Traditional":
        B = float(box_size)
    elif method == "ATR":
        last_atr = float(tv_wilder_atr(df, atr_length).dropna().iloc[-1])
        B = max(mintick, round(last_atr / mintick) * mintick)
    elif method == "Percentage":
        raw_b = float(df["close"].iloc[-1]) * (pct_value / 100.0)
        B = max(mintick, round(raw_b / mintick) * mintick)
    else:
        raise ValueError(f"Bilinmeyen yöntem: {method}")

    p0 = float(df["close"].iloc[0])
    anchor = np.floor(p0 / B) * B
    U, D, direction = anchor + B, anchor, 0
    pend_high, pend_low = p0, p0
    bricks = []

    for ts, row in df.iterrows():
        o, h, l, c = float(row["open"]), float(row["high"]), float(row["low"]), float(row["close"])
        pts = [c] if source == "close" else ([h, l, c] if c < o else [l, h, c])
        for p in pts:
            pend_high, pend_low = max(pend_high, p), min(pend_low, p)
            if direction == 0:
                if p >= U + B - 1e-12:
                    direction, D, U = 1, U, U + B
                    bricks.append({"datetime": ts, "open": D, "high": U, "low": pend_low if wicks else D,
"close": U, "dir": 1, "box": B})
                    pend_high = pend_low = p
                elif p <= D - B + 1e-12:
                    direction, U, D = -1, D, D - B
                    bricks.append({"datetime": ts, "open": U, "high": pend_high if wicks else U, "low": D,
"close": D, "dir": -1, "box": B})
                    pend_high = pend_low = p
            elif direction == 1:
                while p >= U + B - 1e-12:
                    n_open, n_close = U, U + B
                    bricks.append({"datetime": ts, "open": n_open, "high": n_close, "low": min(n_open,
pend_low) if wicks else n_open, "close": n_close, "dir": 1, "box": B})
                    D, U, pend_low = n_open, n_close, n_close
                if p <= D - B + 1e-12:
                    direction, first_rev = -1, True
                    while p <= D - B + 1e-12:
                        n_open, n_close = D, D - B
                        w_hi = max(n_open, pend_high) if (wicks and first_rev) else n_open
                        bricks.append({"datetime": ts, "open": n_open, "high": w_hi, "low": n_close,
"close": n_close, "dir": -1, "box": B})
```

```
                        U, D, first_rev, pend_high = n_open, n_close, False, n_close
            elif direction == -1:
                while p <= D - B + 1e-12:
                    n_open, n_close = D, D - B
                    bricks.append({"datetime": ts, "open": n_open, "high": max(n_open, pend_high) if wicks
else n_open, "low": n_close, "close": n_close, "dir": -1, "box": B})
                    U, D, pend_high = n_open, n_close, n_close
                if p >= U + B - 1e-12:
                    direction, first_rev = 1, True
                    while p >= U + B - 1e-12:
                        n_open, n_close = U, U + B
                        w_lo = min(n_open, pend_low) if (wicks and first_rev) else n_open
                        bricks.append({"datetime": ts, "open": n_open, "high": n_close, "low": w_lo,
"close": n_close, "dir": 1, "box": B})
                        D, U, first_rev, pend_low = n_open, n_close, False, n_close
    return pd.DataFrame(bricks)
```

3.2 Python Referans Uygulaması 2: `request.security_lower_tf()`, Volume Delta ve `request.footprint()`

Aşağıdaki Python modülü; alt zaman dilimi (LTF) veri çerçevesini üst zaman dilimi (HTF) barlarına eşleyerek **`request.security_lower_tf()`** dizilerini üretir, 3 kademeli polarite kuralıyla **Volume Delta OHLC** mumlarını hesaplar ve Pine Script v6 **`request.footprint()`** fonksiyonunun POC, eşitlik bozma kurallı Değer Alanı (VAH/VAL) ve %300 diyagonal dengesizlik (imbalance) çıktısını birebir yeniden üretir:

```
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class VolumeRow:
    down_price: float
    up_price: float
    buy_volume: float = 0.0
    sell_volume: float = 0.0
    has_buy_imbalance: bool = False
    has_sell_imbalance: bool = False

    @property
    def total_volume(self) -> float: return self.buy_volume + self.sell_volume
    @property
    def delta(self) -> float: return self.buy_volume - self.sell_volume

def compute_tv_intrabar_polarity(df_ltf: pd.DataFrame) -> np.ndarray:
    '''TradingView Resmi 3 Kademeli Intrabar Alış (+1) / Satış (-1) Polarite Durum Makinesi.'''
    o = df_ltf["open"].to_numpy(dtype=np.float64)
    c = df_ltf["close"].to_numpy(dtype=np.float64)
    pol = np.ones(len(df_ltf), dtype=np.int8)
    prev_p = 1
    for i in range(len(df_ltf)):
        if c[i] > o[i]: p = 1
        elif c[i] < o[i]: p = -1
        elif i > 0 and c[i] > c[i - 1]: p = 1
        elif i > 0 and c[i] < c[i - 1]: p = -1
        else: p = prev_p
        pol[i] = prev_p = p
    return pol

def calculate_tv_volume_delta_and_footprint(
    df_ltf: pd.DataFrame,
    htf_rule: str = "15min",
    mintick: float = 0.01,
    ticks_per_row: int = 25,
    va_percent: float = 70.0,
    imbalance_percent: float = 300.0
) -> pd.DataFrame:
    '''LTF OHLCV verisinden HTF bazında Volume Delta (O=0, H, L, C) ve Footprint (POC, VAH, VAL, Imbalance)
üretir.'''
    df = df_ltf.copy()
```

```
    df["polarity"] = compute_tv_intrabar_polarity(df)
    df["signed_vol"] = df["volume"] * df["polarity"]
    row_size = ticks_per_row * mintick
    records = []

    for htf_ts, grp in df.resample(htf_rule, label="left", closed="left"):
        if grp.empty:
            continue
        # 1. Volume Delta Mum Hesabı (Open=0, High=Max CumDelta, Low=Min CumDelta, Close=Final CumDelta)
        cum_d = np.cumsum(grp["signed_vol"].to_numpy(dtype=np.float64))
        vd_open = 0.0
        vd_close = float(cum_d[-1])
        vd_high = float(max(0.0, np.max(cum_d)))
        vd_low = float(min(0.0, np.min(cum_d)))

        # 2. Footprint Fiyat Satırları (Rows) İnşası
        b_low, b_high = float(grp["low"].min()), float(grp["high"].max())
        base = np.floor(b_low / row_size) * row_size
        n_rows = max(1, int(np.ceil((b_high - base) / row_size)) + 1)
        rows = [VolumeRow(round(base + i * row_size, 8), round(base + (i + 1) * row_size, 8)) for i in
range(n_rows)]

        for c_val, v_val, pol_val in zip(grp["close"], grp["volume"], grp["polarity"]):
            idx = min(n_rows - 1, max(0, int(np.floor((float(c_val) - base) / row_size))))
            if pol_val == 1: rows[idx].buy_volume += float(v_val)
            else: rows[idx].sell_volume += float(v_val)

        active = [r for r in rows if r.total_volume > 0]
        if not active:
            continue

        # 3. POC ve Value Area (VAH / VAL) Resmi Eşitlik Bozma Algoritması
        poc_i = int(np.argmax([r.total_volume for r in active]))
        target_va = sum(r.total_volume for r in active) * (va_percent / 100.0)
        acc_vol = active[poc_i].total_volume
        up_i, dn_i, vah_i, val_i = poc_i + 1, poc_i - 1, poc_i, poc_i

        while acc_vol < target_va and (up_i < len(active) or dn_i >= 0):
            u_v = active[up_i].total_volume if up_i < len(active) else -1.0
            d_v = active[dn_i].total_volume if dn_i >= 0 else -1.0
            if u_v > d_v: pick_i, go_up = up_i, True
            elif d_v > u_v: pick_i, go_up = dn_i, False
            else:
                # Eşit hacimde POC'ye yakın olan, eşit uzaklıkta ise üstteki satır seçilir
                dist_u = (up_i - poc_i) if up_i < len(active) else 999999
                dist_d = (poc_i - dn_i) if dn_i >= 0 else 999999
                pick_i, go_up = (up_i, True) if dist_u <= dist_d else (dn_i, False)
            if acc_vol + active[pick_i].total_volume > target_va and acc_vol > active[poc_i].total_volume:
                break
            acc_vol += active[pick_i].total_volume
            if go_up: vah_i, up_i = pick_i, up_i + 1
            else: val_i, dn_i = pick_i, dn_i - 1

        # 4. Çapraz (Diyagonal) %300 İmbalans Tespiti
        ratio = imbalance_percent / 100.0
        for i in range(len(active)):
            if i > 0 and active[i - 1].sell_volume > 0 and active[i].buy_volume >= ratio * active[i -
1].sell_volume:
                active[i].has_buy_imbalance = True
            if i < len(active) - 1 and active[i + 1].buy_volume > 0 and active[i].sell_volume >= ratio *
active[i + 1].buy_volume:
                active[i].has_sell_imbalance = True

        records.append({
            "datetime": htf_ts, "vd_open": vd_open, "vd_high": vd_high, "vd_low": vd_low, "vd_close":
vd_close,
            "poc_low": active[poc_i].down_price, "poc_high": active[poc_i].up_price,
            "vah": active[vah_i].up_price, "val": active[val_i].down_price,
```

```
            "buy_imbalances": sum(r.has_buy_imbalance for r in active),
            "sell_imbalances": sum(r.has_sell_imbalance for r in active)
        })
    return pd.DataFrame(records)
```

4. Resmi Belgelerde Belirsiz Kalan Noktalar ve Kapalı Kutu (Proprietary) Davranışlar

TradingView Yardım Merkezi makaleleri ve Pine Script v5/v6 Dil Başvuru Kılavuzu titizlikle tarandığında, platformun kapalı kaynaklı arka uç (backend) grafik ve veri motoruna ilişkin **resmî belgelerde açık matematiksel formülü verilmeyen veya belirsiz bırakılan 8 kritik nokta** tespit edilmiştir. Python'da %100 bit-özdeşliği (bit-identical reproduction) hedeflenirken bu noktaların bilinmesi ve parametrik olarak test edilmesi zorunludur:

No	Belirsiz Kalan Teknik Nokta	Resmî Belgelerdeki Mevcut Tanım	Belgelenmemiş (Undocumented) Boşluk ve Python Modellemesine Etkisi
1	**ATR Renko'da Kutu Boyunun Hangi Barda Dondurulduğu ve Yuvarlama Hassasiyeti**	ATR yönteminin standart mum grafiğindeki ATR değerini hesaplayıp kutu boyutu yaptığı belirtilir (Understanding Renko charts).	ATR değerinin **son kapanmış bardan mı (`barstate.isconfirmed`) yoksa açık olan canlı bardan mı** alındığı ve **`mintick`**'e yuvarlanırken **`round()`** mu yoksa anlamlı basamak kesmesi mi uygulandığı belgelenmemiştir.
2	**`OHLC` Kaynaklı Renko'da Aynı Mum İçinde Hem `High` Hem `Low` Eşiğin Aşılması**	**`OHLC`** kaynağında fitillerin ve tuğlaların **`High`** ve **`Low`** değerlerine dayandığı belirtilir (What do Renko wicks mean?).	Geniş aralıklı tek bir alt mumun **`High`** değeri +1B devam eşiğini, **`Low`** değeri ise −2B dönüş eşiğini aynı anda aşarsa, geçmiş OHLC verisinde önce **`High`**'ın mı yoksa **`Low`**'un mu işlendiği (Broker Emülatöründeki **`|O-H| < |O-L|`** sezgiseli mi yoksa mum yönü **`C ≥ O`** mu kullanıldığı) Renko belgelerinde açıklanmamıştır.
3	**İlk Renko Tuğlasının Başlangıç (Anchor / Seed) Fiyatı Hizalaması**	Tarihsel veri başlangıç noktasının abonelik planına (5K–40K bar) ve zaman dilimi hizalama kurallarına bağlı olduğu belirtilir (Concepts / Repainting).	Grafiğin ilk tarihsel barında (t_0) ilk tuğla sınırının doğrudan C_0 fiyatından mı yoksa kutu boyunun tam katına (**`floor(C_0 / B) * B`**) hizalanarak mı başlatıldığı resmi belgelerde yer almaz; özellikle **`OHLC`** fitilleri **`kutu boyuna bölünebilir sentetik Open/Close`** ifadesi (What do Renko wicks mean?) ikinci yönteme işaret etmektedir.
4	**`ticker.renko()` Fonksiyonunda `Source`, `Wicks` ve `Percentage` Eksikliği**	**`ticker.renko(symbol, style, param)`** fonksiyonu **`"ATR"`** ve **`"Traditional"`** stillerini örnekler (Non-standard charts data).	Grafik arayüzünde seçilebilen **`Source (Close vs OHLC)`** ve **`Percentage (LTP)`** seçeneklerinin **`ticker.renko()`** fonksiyonunda nasıl argüman olarak verileceği veya fonksiyonun varsayılan olarak hangi kaynağı baz aldığı kılavuzda belirsizdir.
5	**Volume Footprint'te Çoklu Satır Kesen Bir Intrabarın Hacim Dağıtımı**	Footprint'in alt zaman dilimi hacimlerini alış/satış olarak ayırıp fiyat satırlarına topladığı belirtilir (Volume footprint guide, Other timeframes and data).	Alt zaman dilimi (ör. 1S veya 1dk) mumunun kendi **`High - Low`** aralığı birden fazla footprint fiyat satırını (**`ticks_per_row`**) keserse, o intrabarın hacminin tamamının intrabarın **`Close`** fiyatındaki satıra mı atandığı yoksa kapsadığı satırlara tekdüze/orantılı olarak mı dağıtıldığı belgelenmemiştir.

No	Belirsiz Kalan Teknik Nokta	Resmî Belgelerdeki Mevcut Tanım	Belgelenmemiş (Undocumented) Boşluk ve Python Modellemesine Etkisi
6	**Footprint Tarihsel Alt Zaman Dilimi Geçiş Sınırlarının (`1T → 1S → 1 → 60`) Belirsizliği**	Tarihsel veri tükendikçe alt zaman diliminin kademeli olarak büyüdüğü (`1T → 1S → 1 → 60`) açıklanır (Volume footprint guide).	Geçmişte tam olarak hangi bar indeksinde **1T**'den **1S**'ye veya **1S**'den **1** dakikaya düşüldüğü sunucu önbelleğine bağlıdır ve Pine Script v6 **`footprint`** nesnesi üzerinde o barın hangi LTF çözünürlüğünden inşa edildiğini döndüren bir metod yoktur.
7	**Footprint `Row Size = Auto` Modundaki `0.2 * NormalizedATR` Tanımı**	Otomatik satır boyutunun **`0.2 * NormalizedATR`** formülüyle hesaplandığı belirtilir (Volume footprint guide).	**`NormalizedATR`** değişkeninin tam matematiksel formülü (**`ta.atr(len) / syminfo.mintick`** değerinin hangi yuvarlama basamağına veya standart 1-2-5-10 tick ızgarasına yuvarlandığı) resmi belgelerde açıkça tanımlanmamıştır.
8	**İlk Intrabarın Doji (`close == open`) Olması Durumunda Başlangıç Polaritesi**	**`close == open`** olduğunda önceki intrabarın kapanışına bakıldığı, o da eşitse önceki intrabarın durumunun korunduğu belirtilir (Volume Delta).	Veri kümesinin en ilk intrabarında (*i = 0*) **`close == open`** gerçekleşirse ve öncesinde hiçbir intrabar yoksa, başlangıç polaritesinin varsayılan olarak **Alış (`+1`)** kabul edildiği yalnızca **`TradingView/ta`** kütüphane kaynak kodundan çıkarılabilmektedir; destek makalesinde bu başlangıç koşulu yazılı değildir.

5. Sonuç, Mimari Sentez ve Doğrulama Önerileri

TradingView resmi belgeleri ve Pine Script v5/v6 başvuru kılavuzu üzerinden yürütülen bu derinlemesine inceleme, algoritmik sistem geliştiricileri için üç temel kuralı ortaya koymaktadır:

1. **Renko Strateji Geri Testlerinde Standart Mum Doğrulaması Zorunludur:** Renko grafiklerinin zamanı sıkıştırması, dönüş tuğlalarında ($2 \times B$) sentetik açılış fiyatını gerçek fiyatın $1 \times B$ gerisine yerleştirmesi ve **`fill_orders_on_standard_ohlc`** özelliğinin Renko'da çalışmaması nedeniyle, doğrudan Renko grafiği üzerinde yapılan strateji geri testleri gerçek piyasa yürütmesini temsil etmez (Concepts / Strategies, Strategy Peeking Into Future). Ayrıca **`ATR`** ve **`Percentage (LTP)`** kutu boyu yöntemleri her yeni barda tüm geçmiş tuğla ızgarasını yeniden boyadığından (repainting), hem Pine Script hem de Python testlerinde kutu boyu başlangıçta bir kez ATR ile ölçülüp **Traditional (Sabit)** değer olarak kilitlenmelidir (Understanding Renko charts, ATR Based Renko Brick Size).
2. **Bar İçi Emir Akışı İçin `request.security_lower_tf()` ve `request.footprint()` Ayrımı:** Tüm abonelik planlarında çalışan taşınabilir bir Volume Delta modeli gerektiğinde **`request.security_lower_tf()`** ile 3 kademeli polarite durum makinesi kullanılmalı; fiyat kademesi bazında POC, %70 Değer Alanı (VAH/VAL) ve %300 diyagonal dengesizlik (imbalance) analizi gerektiğinde ise Pine Script v6'nın yerel **`request.footprint()`** mimarisi tercih edilmelidir (Concepts / Other timeframes and data, Volume Delta).
3. **Python'da Birebir Yeniden Üretim Stratejisi:** Standart Pine Script v6 indikatör, strateji ve **`request.security_lower_tf()`** mantığını Python'da %99,77+ bit-özdeşliğiyle çalıştırmak için **PyneCore** (PyneCore Compatibility); tick verisinden hızlı fitilli Renko üretimi için **renkodf** (renkodf PyPI); TradingView'ın OHLC mum bazlı Wilder RMA-ATR Renko motorunu ve **`request.footprint()`** POC/VA/Imbalance algoritmasını sıfır sapmayla simüle etmek için ise bu raporda sunulan referans Python modülleri doğrudan kullanılmalıdır.

Resmî Başvuru Kaynakları ve Dizin

- TradingView Help Center — Understanding Renko charts (Brick calculation methods: ATR, Traditional, Percentage LTP, Projection bars)
- TradingView Help Center — What do Renko wicks mean? (1x continuation, 2x reversal, Close vs OHLC wicks)
- TradingView Help Center — Custom chart intervals — personalizing your analysis (Renko, Kagi, and P&F price intervals)
- TradingView Help Center — Strategy produces unrealistically good results by peeking into the future (Japanese-style candles & 4 transactions)
- TradingView Help Center — Volume Delta — Built-in Indicators (Auto LTF selection rules & intrabar polarity algorithm)
- TradingView Help Center — Volume footprint charts: a complete guide (LTF fallback chain, 300% diagonal imbalance, Auto ATR row size)
- TradingView Help Center — Volume profile indicators: basic concepts (POC & Value Area VAH/VAL tie-breaking algorithm)
- Pine Script® v6 User Manual — Concepts / Non-standard charts data (ticker.renko, ticker.heikinashi, ticker.pointfigure)
- Pine Script® v6 User Manual — Concepts / Other timeframes and data (request.security_lower_tf & request.footprint)
- Pine Script® v6 User Manual — Language / Type system (footprint and volume_row reference types and methods)
- Pine Script® v6 User Manual — Concepts / Strategies (Broker emulator assumptions & non-standard chart backtesting)
- Pine Script® v6 User Manual — Concepts / Repainting (Historical vs realtime fluid data & LTF repainting)
- Pine Script® v6 User Manual — Writing scripts / Limitations (100K/125K/200K intrabar limits & single request.footprint limit)
- PyneCore Open-Source Runtime — Pine Script v6 Compatibility, request.security() & LTF Internals, PyneSys/pynecore GitHub
- Python Open-Source Renko Packages — srlcarlg/renkodf GitHub, ChillarAnand/stocktrends GitHub, matplotlib/mplfinance